

Operadores

Aritméticos: + (adição), - (subtração), * (multiplicação), / (divisão), // (divisão inteira), % (resto), ** (exponenciação)

Comparação: == (igual), != (diferente), > (maior), < (menor), >= (maior ou igual), <= (menor ou igual)

Lógicos: and, or, not

Pertinência: in, not in

Escrita

Interpolação de strings: print(f"Olá, {nome}!")

Múltiplos argumentos: print("Nome:", nome, "Idade:", idade)

Controlo de Fluxo

Ciclo if, else e elif Usados para executar diferentes blocos de código com base em condições.

Ciclo while: Executa um bloco de código repetidamente enquanto a condição for verdadeira.

```
while condição:
    # Código repetido
```

Ciclo for: Itera sobre uma sequência (lista, tuplo, string, etc.) ou um intervalo de números.

```
for i in range(5): # Vai de 0 até 4
    print(i)
    o break: Sai imediatamente do ciclo.
    o continue: Passa para a próxima iteração
```

Funções

Sintaxe:

```
def nome_da_funcao(parametros_opcionais):
    # Bloco de código da função
    return valor_opcional
```

Recursão: A recursão ocorre quando uma função se chama diretamente dentro de seu próprio corpo.

```
def fatorial(n):
    if n == 0: # Caso base return 1 else:
    return n * fatorial(n- 1) # Chamada recursiva.
```

Operações:

Concatenar: "Olá, " + "Mundo" → "Olá, Mundo"

Repetir: "Oi! " * 3 → "Oi! Oi! Oi! "

Tamanho: len("Python") → 6

Indexação: "Python"[0] → 'P', "Python"[-1] → 'n'

Fatiamento: "Python"[0:3] → 'Pyt', "Python"[:2] → 'Pt', "Python"[:-1] → 'nohtyP'

Métodos úteis:

Transformar: .lower(), .upper(), .strip(), .replace(a, b)

Dividir e juntar: .split(), .join(lista)

Procurar e contar: .find("x"), .count("x")

- split(): Divide uma string em partes com base em um delimitador.

```
texto = "Olá Mundo"
palavras = texto.split() # ['Olá', 'Mundo']
```

- strip(): Remove espaços no começo e no final da string.

```
texto = " Olá Mundo! "
print(texto.strip()) # "Olá Mundo!"
```

- find(): Retorna o índice da primeira ocorrência de uma substring.

```
texto = "Olá Mundo"
print(texto.find("Mundo")) # 4
```

- replace(): Substitui uma substring por outra.

```
texto = "Olá Mundo"
print(texto.replace("Mundo", "Pyt")) # "Olá Pyt"
```

Fatiamento (slice) é usado para acessar partes de uma string, lista ou tupla.

```
texto = "Python"
print(texto[1:4]) # "yth"
```

Verificação:

isalpha() → Apenas letras | isdigit() → Apenas números | isalnum() → Letras ou números

Tuplos

Sequências imutáveis de elementos, criadas com parênteses ().

Isso torna as tuplas mais rápidas e seguras para dados que não devem ser modificados.

```
tupla = (1,)
```

Operações:

Indexação e Fatiamento, semelhantes às strings.

Métodos úteis: .index(x), .count(x)

Tuplos Aninhados:

Tuplos podem conter outros tuplos ou listas.

```
tuplo = (ele1, (ele1, ele2), ["Python", "Java"])
```

Listas

Sequências mutáveis de elementos, criadas com colchetes [].

Operações:

Tamanho: len(lista)

Indexação e Fatiamento: Igual aos tuplos e strings.

Modificar Listas:

Adicionar: .append(), .insert()

Remover: .remove(), .pop()

lista[indice] = novo_valor

Métodos Comuns:

```
.count(x), .index(x), .sort(), .reverse()
```

Listas Aninhadas:

Podem conter outras listas ou tuplos.

Listas por Compreensão:

Forma compacta para criar listas.

Sintaxe: nova_lista = [expressão for item in iterable if condição]

quadrados = [x**2 for x in range(5)] # Saída: [0, 1, 4, 9, 16]

Ficheiros

Permitem armazenar e recuperar dados.

ex:

```
with open('arquivo.txt', 'r') as file:
    conteudo = file.read()
```

Modos de abertura:

'r': leitura

'w': escrita (sobrescreve o conteúdo existente)

'a': adicionar (escreve no final do ficheiro)

Ler ficheiros:

.read(): Lê todo o conteúdo.

.readline(): Lê uma linha de cada vez.

.readlines(): Lê todas as linhas e retorna uma lista.

Escrever ficheiros:

.write(): Escreve uma string.

.writelines(): Escreve uma lista de strings.

Exceções

As **exceções** são erros que ocorrem durante a execução de um programa. O Python permite capturar e tratar esses erros usando blocos try e except.

```
try:
    x = 10 / 0 # Erro de divisão por zero
except ZeroDivisionError:
    print("Não é possível dividir por zero!")
```

Exceções comuns:

- ValueError: Erro ao passar um valor errado para uma função.
- IndexError: Tentativa de acessar um índice fora do intervalo da lista.

#-----PROBLEMAS-----

#Lê dados de um ficheiro e cria uma lista de dicionários, onde cada dicionário contém informações de altura, peso, sexo e idade

```

def ler_dados(ficheiro):
    lista_dados = []
    with open(ficheiro, 'r') as file:
        linhas = file.readlines()
        chaves = linhas[0].strip().split()
        for linha in linhas[1:]:          # Percorre as restantes linhas para criar os
dicionários
            valores = linha.strip().split()
            altura = float(valores[0])
            peso = int(valores[1])
            sexo = valores[2]
            idade = int(valores[3])
            dado = {chaves[0]: altura, chaves[1]: peso, chaves[2]: sexo, chaves[3]:
idade}
            lista_dados.append(dado)
    return lista_dados
#-----PROBLEMAS-----
#verifica se um número ppp é um número de Fermat, ou seja, da forma 2^(2n)+1
def fermat(p):
    n = 0
    while True:
        fermat_num = 2**(2*n) + 1
        if fermat_num == p:
            return True
        elif fermat_num > p:
            return False
        n += 1
#print(fermat(257))
#print(fermat(20))
#-----PROBLEMAS-----
#verifica se existem dois números em um ficheiro cujas somas são iguais a um valor nnn.
def soma_n(ficheiro,n):
    numeros = []
    with open(ficheiro, 'r') as file:
        for linha in file:
            numeros.append(int(linha.strip()))
    for x in range(len(numeros)):
        for y in range(x + 1, len(numeros)):
            if numeros[x] + numeros[y] == n:
                return True
    return False
#---problema A -----
#calcula a distância Euclidiana entre os pontos correspondentes nas duas listas l1 e l2
import math
def calcular_distancias(l1, l2):
    distancias = []
    for x, y in zip(l1, l2):
        distancia = math.sqrt(math.pow(x, 2) + math.pow(y, 2))
        distancias.append(distancia)
    return distancias
l1 = [3, 1, 4, 6]
l2 = [4, 1, 0, 8]
#-----PROBLEMAS -----
#inverte as palavras de uma frase, mas mantém as palavras especificadas na lista lst inalteradas
def inverte_seguro(str, lst):
    resultado = ""
    for i in str.split():
        if i in lst:
            resultado += i + " "
        else:
            resultado += i[::-1] + " "
    return resultado.strip()
frase = "as coisas da que o meu corpo se lembra"
protegidas = ["coisas", "lembra"]
#print(inverte_seguro(frase, protegidas))

```

```
#-----
```

#verifica se dois números n e m são primos entre si (ou seja, se o maior divisor comum entre eles é 1)

```
def coprimos(n, m):
    d = 2
    while d <= min(n, m):
        if n % d == 0 and m % d == 0:
            return False
        d += 1
    return True
#print(coprimos(4, 9))#True, pois 4 e 9 são coprimos
```

```
#-----
```

#recebe uma frase, e para cada palavra, concatena a palavra com sua versão invertida, gerando uma nova string

```
frase = input('Escreva uma frase: ')
lista_palavras = frase.split()
s = ""
for pal in lista_palavras:
    s = s+pal+pal[::-1]+" "
print(s[:-1])
```

```
#-----
```

#ela soma os valores associados a cada chave (nome) em uma lista de tuplas

```
def soma_t(lis):
    resultado = {}
    for nome, numero in lis:
        if nome in resultado:
            resultado[nome] += numero
        else:
            resultado[nome] = numero
    return resultado
lista = [('a', 10), ('b', 20), ('c', 30), ('a', 11), ('c', 44)]
print(soma_t(lista))# Saída esperada: {'a': 21, 'b': 20, 'c': 74}
```

```
#-----
```

#buscando o segundo maior valor da lista

```
def segundo_maior(ls):
    ls_copia = ls[:]
    ls_copia.remove(max(ls))
    return max(ls_copia)
```

```
#-----
```

#Inteiro positivo n devolve n!. Escreva em Python uma nova função combinac(p, q) que dados os inteiros positivos p e q (com $p \geq q$) devolve o inteiro que representa as combinações sem repetição de p, q a q, ou seja $p!/(q! (p-q)!)$

```
def combinac(p, q):
    return factorial(p)/(factorial(q) * factorial(p-q))
```

```
#-----
```

#uma frase e imprima a última palavra invertida

```
frase = input("Escreva uma frase: ")última = frase.split()[-1]
print(str(frase.rfind(ultima))+": "+ultima[::-1])
```

```
#-----
```

def sublst_prox(dados, x, delta) :#devolva a sublista de elementos de dados que estão na vizinhança de x a menos de delta

```
sublista = []
for numero in dados:
    if abs(numero-x) < delta:
        sublista.append(numero)
return sublista
```

```
#-----
```

#envolva uma lista de tuplos em que o primeiro elemento corresponde ao nome do parâmetro e o segundo elemento ao seu respectivo valor

```
def obter_parametros_tuplos(nome_ficheiro):
    ficheiro = open(nome_ficheiro, 'r')
    linhas_ficheiro = ficheiro.readlines()
    ficheiro.close()
    lista_tuplos = []
    for linha in linhas_ficheiro:
        elementos = linha.split(' = ')
        lista_tuplos.append((elementos[0], int(elementos[1])))
    return lista_tuplos
```

```

#-----
#dada uma lista de triplos da forma (int, str, str), determine se o primeiro elemento de cada triplo é a soma dos
tamanhos das duas strings no mesmo triplo.
def valida_tamANHos(lista):
    for t in lista:
        if t[0] != len(t[1]) + len(t[2]):
            return False
        return True
#-----

#que recebe uma lista de strings e um separador, e retorna uma única string concatenada, onde as palavras são
separadas pelo separador fornecido
def junta(palavras, separador):
    return separador.join(palavras)
    print(junta(['Este', 'exame', 'é', 'fácil.'], '-')) #Este-exame-é-fácil.'
#-----

#verifica se uma matriz quadrada de números inteiros é um quadrado mágico:
def quadrado_magico(matriz):
    soma_ref = sum(matriz[0])
    for linha in matriz:
        if sum(linha) != soma_ref:
            return False
    return True
matriz_boa = [[2, 7, 6], [9, 5, 1], [4, 3, 8]]
print(quadrado_magico(matriz_boa)) # True
#-----

#Recebe o nome de um ficheiro e cria um novo ficheiro double.txt, que é uma cópia do recebido.
def funcao (ficheiro):
    """
    Recebe o nome de um ficheiro e cria um novo ficheiro
    double.txt, que é cópia do recebido.
    Requires: ficheiro é uma String com o nome de um ficheiro
    Ensures: Ficheiro double.txt com a cópia do ficheiro recebido
    """
    in_file = open("myFile.txt", 'r')
    in_data = in_file.read()

    out_file = open("double.txt", 'w')
    out_file.write(in_data)

    in_file.close()
    out_file.close()
#-----

#calcula a área de um quadrado se receber 1 argumento ou a área de um retângulo se receber 2 argumentos
def area_flexivel(*args):
    if len(args) == 1:
        return area_quad(args[0])#1 arg, área-quad
    elif len(args) == 2:
        return area_rect(args[0],args[1])#2 arg área-ret

def area_rect(comprimento, largura):
    return comprimento * largura

def area_quad(lado):
    return area_rectangulo(lado, lado)
#-----

#recebe um número base a e um número n(expoente máximo)e retorna uma lista com as potências de a de 0 até
n(inclusive)
def lista_potencias(a, n):
    return [a**i for i in range(n+1)]
#-----

#cria uma nova matriz onde cada elemento interno é substituído pela soma dos seus 8 vizinhos, e os elementos das
bordas mantêm seus valores originais
def soma_vizinhos(matriz):
    m = len(matriz) # Número de linhas
    n = len(matriz[0]) # Número de colunas

    #Criar uma nova matriz com as mesmas dimensões

```

```

resultado = []
for i in range(m):
    resultado.append([0] * n)
for i in range(m):
    for j in range(n):
        #borda, copia o valor original
        if i ==0 or i== m-1 or j==0 or j==n-1:
            resultado[i][j] = matriz[i][j]
        else:
            #Caso contrário, soma os 8 vizinhos
            soma = 0
            for x in range(i - 1, i + 2):
                for y in range(j - 1, j + 2):
                    soma += matriz[x][y]
            soma -= matriz[i][j]#- o valor próprio elem.
            resultado[i][j] = soma
    return resultado
matriz = [[3, 1, 10, 2,-1], [0, 4, 2, 0, 5], [8, 8, 0, 9, 4], [1, 1, 1, 1, 1]]
print(soma_vizinhos(matriz))
#-----

```